

Slices

Thursday, 10 September 2026

8:32 AM

* Array in Go:

Array are fixed-size groups of variables of the same type.

```
var myInts [10]int
```

```
primes := [6]int{2, 3, 5, 7, 11, 13}
```

* Slices in Go:

Array are fixed in size, once you make an array like [10]int you can't add an 11th element.

- A slice is a dynamically-sized, flexible view of the elements of an array.
- The zero value of slice is nil
- Non-nil slices always have an underlying array, though it isn't always specified explicitly. To explicitly create a slice on top of an array we can do:

```
primes := [6]int{2, 3, 5, 7, 11, 13}
mySlice := primes[1:4]
// mySlice = {3, 5, 7}
```

The syntax is:

```
arrayname[lowIndex:highIndex]
arrayname[lowIndex:]
arrayname[:highIndex]
arrayname[:]
```

- where lowIndex is inclusive and highIndex is exclusive.
- lowIndex, highIndex or both can be omitted to use the entire array on that side of the colon:

* Slices Review:

Slices wrap arrays to give a more general, powerful, and convenient interface to sequences of data. Except for items with explicit dimensions such a transformation matrices, most array programming in Go is done with slices rather than simple arrays.

- slices hold references to an underlying array, and if you assign an slice to another, both refer to the same array.
- if function takes a slice argument, any changes it makes to the elements of the slice will be visible to the caller, analogous to passing a pointer to the underlying array.

* Make:

Most of the time we don't need to think about the underlying array of slice. we can create a new slice using the make function.

```
mySlice := make([]int, 5, 10)
```

- slices created with make will be filled with the zero value of the type.

* Length:

The length of a slice is simply the number of elements it contains it is accessed using the built-in len() function:

```
len(mySlice) = 3
```

* Capacity:

The capacity of a slice is the number elements in the underlying array, counting from the first element in the slice. it is accessed using the built-in cap() function:

```
cap(mySlice) = 10
```

* Len and Cap Review:

The length of a slice may be changed as long as it still fits within the limits of the underlying array; just assign it to a slice of itself.

- The capacity of a slice, accessible by the built-in function cap, reports the maximum length the slice may assume.
- The function uses the fact that len and cap are legal when applied to the nil slice, and return 0.

* Variadic:

A variadic function takes an arbitrary number of final arguments. These are called variadic arguments and are received as a slice using the ... syntax in the function signature.

```
func concat(strs ...string) string {
    // ...
}
```

* Spread Operator:

The spread operator allows us to pass a slice into variadic function. The spread operator consists of three dots following the slice in the function call.

```
names := []string{"bob", "sue", "alice"}
printStrings(names...)
```

* Append:

The build-in append function is used to dynamically add elements to a slice.

```
func append(slice []Type, elems ...Type) []Type
```

- If the underlying array is not large enough, append() will create a new underlying array and point the returned slice to it.

```
slice = append(slice, oneThing)
slice = append(slice, first, second)
slice = append(slice, anotherSlice...)
```

* Range:

Go provides syntactic sugar to iterate easily over elements of a slice.

```
for index, ELEMENT := range SLICE {
    // ...
}
```

The element is a copy of the value at that index.

* Slice of slices:

slices can hold other slices, effectively creating matrix, or a 2D slice.

```
rows := [][]int{}
rows = append(rows, []int{2, 2, 3})
rows = append(rows, []int{4, 5, 6})
fmt.Println(rows)
```

* Tricky slices:

The append() function changes the underlying array of it's parameter AND returns a new slice. This means that using append() on anything other than itself is usually a BAD idea.

```
// don't do this!
```

```
someSlice = append(otherSlice, element)
```